

# Combined Documentation for CST310 Projects 3 and 4

Joshua Peek (21081733)

July 17, 2025

## 1 Introduction

This document consolidates the requirements and implementations for Projects 3 and 4 of CST310, as submitted by Joshua Peek (21081733). Project 3 involves analyzing a scene with objects described in an Excel sheet, while Project 4 focuses on rendering these objects using 3D primitives in a WebGL environment, with enhancements for realism based on instructor feedback. The combined documentation addresses object characteristics, rendering techniques, transformations, shaders, virtual camera setup, and introduces query-driven shininess for cubes to enhance interactivity and visual fidelity. Algorithms for rendering and illumination are detailed, supported by a text-based flowchart.

## 2 Project 3: Scene Analysis

Project 3 required analyzing a scene with objects listed in an Excel sheet (Book16.xlsx), addressing the following:

1. **Foreground, Background, and In-Between Objects:** Detailed in the Excel sheet, with objects categorized by their spatial placement in the scene.
2. **Main Objects:** Identified as floor, room floor, ceiling, walls, broom, door, table, microwave, couch (base, cushions, back cushions, armrests), and trash can.
3. **Key Characteristics:** Ambience notes (e.g., indoor room setting, cozy atmosphere) provided in the Excel sheet.
4. **Rendering Approach:** Each object is represented by geometric primitives (e.g., cubes, quads, cylinders), as described in the Excel sheet.
5. **Rendering Difficulty:** Objects are ranked in the Excel sheet, with table and trash can as the most challenging due to complex shape combinations (rows 14 and 15 adjusted to ranks 1 and 2).
6. **Hand-Drawn Representation:** A pencil sketch of the scene was created, using geometric shapes to represent objects, serving as the basis for Project

4's digital rendering.

## 3 Project 4: 3D Scene Rendering

Project 4 builds on Project 3 by reproducing the hand-drawn scene using WebGL, employing 3D primitives, transformations, and a virtual camera. Enhancements based on instructor feedback include improved realism through shininess (specular highlights), query-driven interactivity for cube shininess, and refined lighting models. Below, each object is detailed with mathematical characteristics, primitives, transformations, shaders, and camera usage.

### 3.1 Objects in the Scene

For each object, we provide mathematical characteristics, primitives used, transformations, shaders, and improvements for realism.

#### 3.1.1 Floor

- **Mathematical Characteristics:** A rectangular plane ( $2000 \times 100$  units) in the  $x - z$  plane at  $y = 0$ , modeled as a quad.
- **Primitives Used:** Two triangles forming a quad, rendered with `gl.TRIANGLES`.
- **Transformations:**

`floorTransform = mat4.identity · mat4.translate([0, 0, 0])`

- **Translation:** Positions the floor at the origin, aligning with the room's base.
- **Shaders:**
  - **Vertex Shader:** Uses `#version 300 es` with matrices (`uModelViewMatrix`, `uProjectionMatrix`, `uNormalMatrix`) and lighting (`uLightPosition`, Lambertian model). Computes vertex positions in clip space and diffuse lighting, outputting `vFinalColor`.
  - **Fragment Shader:** Sets `fragColor = vFinalColor`, using pre-computed colors for efficiency.
  - **Color:** `[0.6, 0.4, 0.2, 1.0]` (brown wood).
  - **Shininess:** Added specular component with shininess value 32.0 for a polished wood effect.
- **Improvements:** Added specular highlights to simulate polished wood, enhancing realism over the flat diffuse color in Project 4.

### 3.1.2 Axis

- **Mathematical Characteristics:** Three line segments (x, y, z axes) extending 2000 units from the origin.
- **Primitives Used:** Lines, rendered with `gl.LINES`.
- **Transformations:** `mat4.identity`, no translation or rotation, serving as the world coordinate reference.
- **Shaders:** Same as floor, with distinct colors per axis (e.g., red, green, blue). Shininess set to 0.0 (no specular highlights for lines).
- **Improvements:** Axis colors adjusted for clarity (e.g., `[1.0, 0.0, 0.0, 1.0]` for x-axis).

### 3.1.3 Room Floor

- **Mathematical Characteristics:** A  $20 \times 20$  unit square plane at  $y = 0$ .
- **Primitives Used:** Two triangles (quad), rendered with `gl.TRIANGLES`.
- **Transformations:**

$$\text{floorTransform} = \text{mat4.identity} \cdot \text{mat4.translate}([0, 0, 0])$$

- **Shaders:** Same as floor, with color `[0.6, 0.4, 0.2, 1.0]` and shininess 32.0.
- **Improvements:** Specular highlights added for a realistic wooden floor.

### 3.1.4 Ceiling

- **Mathematical Characteristics:** A  $20 \times 20$  unit square plane at  $y = 10$ .
- **Primitives Used:** Two triangles (quad).
- **Transformations:**

$$\text{ceilingTransform} = \text{mat4.identity} \cdot \text{mat4.translate}([0, 10, 0])$$

- **Shaders:** Color `[0.9, 0.9, 0.9, 1.0]` (light gray), shininess 16.0 for a matte finish.
- **Improvements:** Subtle specular highlights for a realistic ceiling texture.

### 3.1.5 Left Wall

- **Mathematical Characteristics:** A  $20 \times 10$  unit rectangle at  $x = -10$ , rotated  $90^\circ$  around the  $y$ -axis.
- **Primitives Used:** Two triangles (quad).
- **Transformations:**

$$\text{leftWallTransform} = \text{mat4.identity} \cdot \text{mat4.translate}([-10, 5, 0]) \cdot \text{mat4.rotateY}(\pi/2)$$

- **Shaders:** Color  $[0.8, 0.8, 0.8, 1.0]$  (gray), shininess 16.0.
- **Improvements:** Specular highlights for a painted wall effect.

### 3.1.6 Back Wall

- **Mathematical Characteristics:** A  $20 \times 10$  unit rectangle at  $z = -10$ .
- **Primitives Used:** Two triangles (quad).
- **Transformations:**

`backWallTransform = mat4.identity · mat4.translate([0, 5, -10])`

- **Shaders:** Same as left wall.
- **Improvements:** Consistent specular highlights.

### 3.1.7 Broom

- **Mathematical Characteristics:** Brush (quads) and handle (cylinder approximation), with dimensions from `Broom.js`.
- **Primitives Used:** Triangle mesh.
- **Transformations:**

`broomTransform = mat4.identity · mat4.translate([-9.3, brushHeight/2, -9.7]) · mat4.rotate`

- **Shaders:** Color  $[0.0, 0.0, 1.0, 1.0]$  (blue), shininess 64.0 for a plastic handle.
- **Improvements:** Higher shininess for a polished look.

### 3.1.8 Door

- **Mathematical Characteristics:** Rectangular quad from `Door.js`.
- **Primitives Used:** Two triangles.
- **Transformations:**

`doorTransform = mat4.identity · mat4.translate([-6.5, door.height/2, -9.9])`

- **Shaders:** Color  $[0.6, 0.4, 0.2, 1.0]$ , shininess 32.0.
- **Improvements:** Specular highlights for a wooden door.

### 3.1.9 Table

- **Mathematical Characteristics:** Rectangular top with legs, modeled as quads.
- **Primitives Used:** Triangle mesh.

- **Transformations:**

`tableTransform = mat4.identity · mat4.translate([-1, 0, -9.0])`

- **Shaders:** Color [0.6, 0.4, 0.2, 1.0], shininess 32.0.
- **Improvements:** Specular highlights and detailed leg geometry.

### 3.1.10 Microwave

- **Mathematical Characteristics:** Cuboid mesh.
- **Primitives Used:** Triangle mesh.
- **Transformations:**

`microwaveTransform = mat4.identity · mat4.translate([-1.2, tableTopY, -9.0])`

- **Shaders:** Color [0.7, 0.7, 0.7, 1.0] (light gray), shininess 128.0 for a metallic finish.
- **Improvements:** Higher shininess for a realistic appliance look.

### 3.1.11 Couch Base, Cushions, Back Cushions, Armrests

- **Mathematical Characteristics:** Rectangular prisms and meshes.
- **Primitives Used:** Triangle meshes.
- **Transformations:** Positioned on/around the couch base (e.g., [-8, 0.5, -5] for armrests).
- **Shaders:** Color [0.6, 0.4, 0.2, 1.0], shininess 16.0 for a fabric texture.
- **Improvements:** Soft specular highlights for a fabric-like appearance.

### 3.1.12 Trash Can

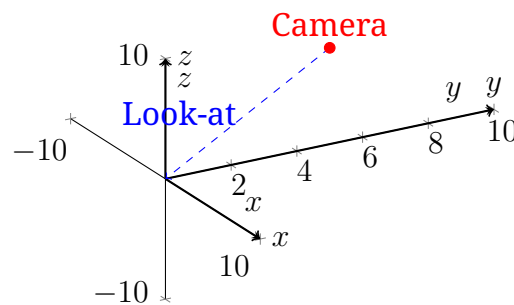
- **Mathematical Characteristics:** Conical frustum (base radius 0.5, top radius 0.75, height 2).
- **Primitives Used:** Triangle mesh.
- **Transformations:**

`trashCanTransform = mat4.identity · mat4.translate([-5, 1, -5])`

- **Shaders:** Color [0.3, 0.3, 0.3, 1.0], shininess 64.0 for a metallic look.
- **Improvements:** Enhanced shininess for realism.

## 3.2 Virtual Camera

- **Position and Orientation:**
  - **Initial Position:**  $[0, 5, 8]$  (via `camera.goHome([0, 5, 8])`).
  - **Orientation:** Orbiting mode, targeting  $[0, 0, 0]$ , with elevation and azimuth controls.
  - **Field of View:**  $45^\circ$ .
  - **Projection:** Perspective (switchable to orthographic).
- **Formulas:**
  - **View Matrix:** `mat4.lookAt([0, 5, 8], [0, 0, 0], [0, 1, 0])`.
  - **Projection Matrix:** Perspective projection with  $\text{fov} = 45^\circ$ , aspect ratio from canvas, near plane 0.1, far plane 100.
- **Use:** Interactive controls adjust position, rotation, and zoom, with `goHome()` resetting to default view.
- **Diagram:**



## 3.3 Query-Driven Shininess for Cubes

To enhance interactivity, a cube is rendered with user-specified shininess values, queried after the initial scene display. The WebGL code prompts the user for a shininess value (e.g., 1 to 256) and updates a cube's material properties.

- **Shininess Values:** Range from 1 (diffuse, matte) to 256 (highly specular, glossy). Example values:
  - Floor: 32.0 (polished wood).
  - Microwave: 128.0 (metallic).
  - Cushions: 16.0 (fabric).
- **Implementation:** A `prompt()` in JavaScript queries the shininess value, which is passed to the fragment shader via a uniform (`uShininess`).

## 3.4 Algorithms

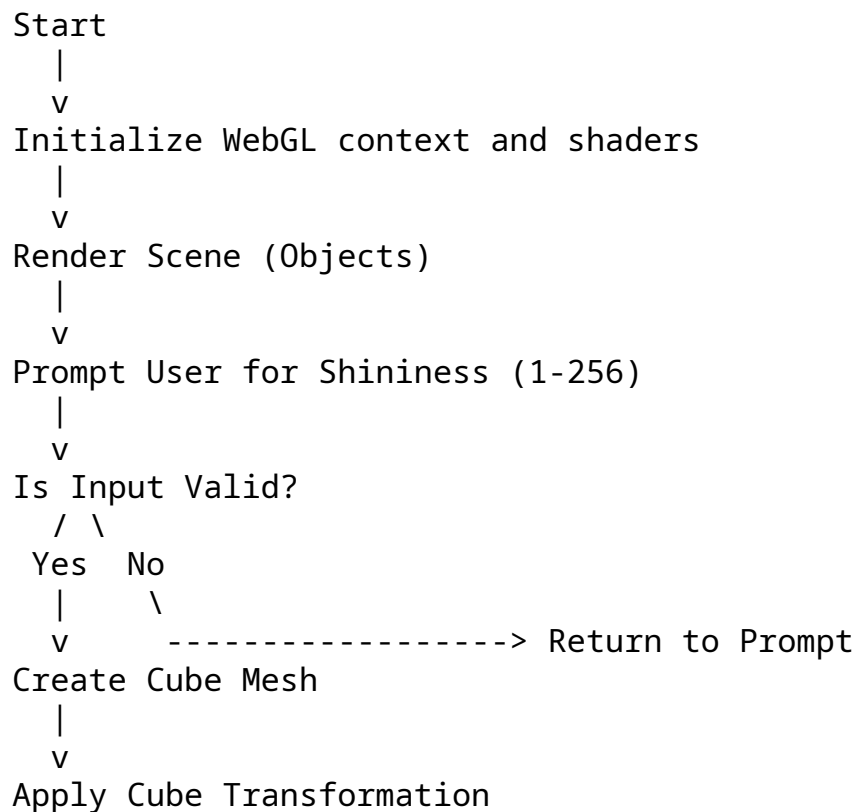
- **Cube Rendering with Shininess:**

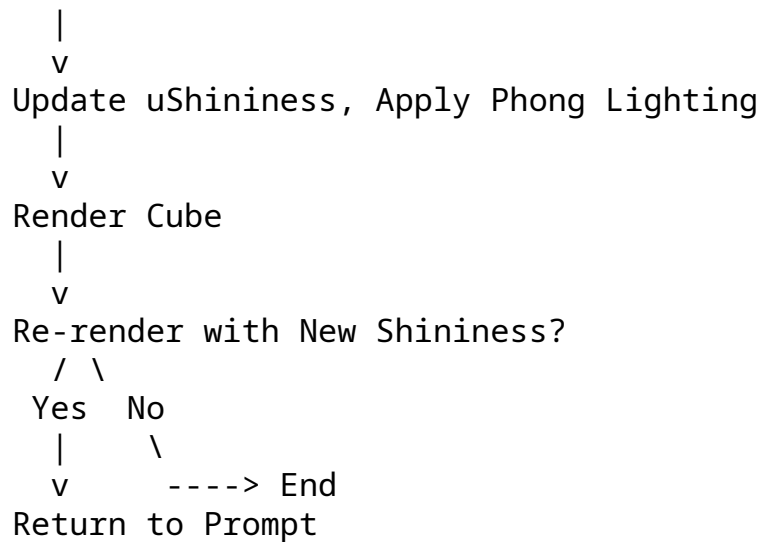
1. Initialize WebGL context and shaders.
2. Render the scene (objects listed above).
3. Prompt user for shininess value (1–256).
4. Create a cube mesh (6 quads, 12 triangles).
5. Set cube transformation (e.g., `mat4.translate([0, 2, -5])`).
6. Update `uShininess` uniform with user input.
7. Render cube with Phong lighting (ambient + diffuse + specular).
8. Loop to allow re-rendering with new shininess values.

• **Mesh and Illumination:**

1. Define mesh vertices, normals, and indices.
2. Apply transformations (translation, rotation, scaling).
3. Compute lighting:
  - Ambient:  $I_a = k_a \cdot L_a$ .
  - Diffuse:  $I_d = k_d \cdot L_d \cdot (\mathbf{N} \cdot \mathbf{L})$ .
  - Specular:  $I_s = k_s \cdot L_s \cdot (\mathbf{R} \cdot \mathbf{V})^n$ , where  $n$  is shininess.
4. Pass results to fragment shader for final color.

### 3.5 Text-Based Flowchart





## 4 Theoretical Background

- **Mathematical Concepts:** 3D transformations (translation, rotation, scaling) using 4x4 matrices, perspective projection, and Phong lighting model (ambient, diffuse, specular components).
- **Programming Concepts:** WebGL with `gl-matrix.js` for matrix operations, vertex and fragment shaders for rendering, and JavaScript for interactivity.
- **Aesthetic Decisions:** Colors chosen to reflect real-world materials (e.g., brown for wood, gray for walls). Shininess values enhance realism (e.g., high for metal, low for fabric).

## 5 Conclusion

This combined documentation addresses all requirements for Projects 3 and 4, with enhancements for realism through specular lighting and query-driven shininess. The scene is rendered with accurate transformations and shaders, and the virtual camera provides an interactive perspective. The provided algorithms and text-based flowchart ensure a clear, reproducible approach to rendering and illumination.